NavSphere firmware technical specification

The supplied project was an ESP-IDF CMake skeleton with an empty `app_main()`. It had no Wi-Fi code, target selection, pin map, SDK version or credentials. The existing CMake project is extended in native C11 and FreeRTOS. ESP-IDF 6.0.2 is installed on this machine; managed MQTT and cJSON dependencies are locked in `dependencies.lock`. No Arduino migration was performed.

Confirmed board and analog feasibility

The user identified an ESP32S/WROOM and selected **six on-board ADC inputs only, without extra analog hardware**. Classic ESP32 ADC2 cannot provide reliable analog acquisition alongside Wi-Fi. Six ADC1 pins are commonly exposed on WROOM development boards; ADC1 GPIO37/38 are not generally exposed.

Logical channel	Physical ADC mapping
analog_1	ADC1 GPIO32
analog_2	ADC1 GPIO33
analog_3	ADC1 GPIO34
analog_4	ADC1 GPIO35
analog_5	ADC1 GPIO36 / VP
analog_6	ADC1 GPIO39 / VN

The JSON always includes exactly six channels and six raw counts. Unavailable channels have zero numeric fields and a cleared validity-mask bit: **zero is not a measurement**. The complete valid mask is 63. A floating ADC pin is electrically readable but has no useful sensor measurement: connect signals and a common ground before interpreting readings. There are no simulated measurements in firmware. Simulator data originates in the separate simulator.

Resolution is 12 bit, attenuation 12 dB, with four configurable averaged samples. The ESP32's useful calibrated range at high attenuation is narrower than its 3.3V supply (consult the exact chip datasheet, commonly about 150-2450mV). Attenuation does not make a pin 5V tolerant. Keep inputs within both the documented ADC measuring range and absolute pad limits; use suitable dividers, buffers and protection. GPIO34/35/36/39 are input only and lack internal pulls. This driver requires eFuse calibration data and marks all channels unavailable when it cannot create a calibration handle. It does not claim assumed-Vref values are calibrated. Per-channel read/calibration failures clear that bit.

A/B output mapping

The user authorized physical GPIO HIGH/LOW controls. A is GPIO25 and B is GPIO26, active high. For an external LED use GPIO25 -> series resistor (for example 1kOhm) -> LED anode, cathode -> GND. GPIO26 can drive a second LED with its own resistor or a logic-level input within ESP32 pad current/voltage limits. No built-in LED mapping is assumed. Do not drive motors, relays or loads directly. Use external pull-downs if LOW must be guaranteed during reset/boot.

Both latches initialize LOW. They reset LOW within one publish cycle (nominal 100ms) after observed Wi-Fi/MQTT disconnect or clock invalidity, and after a configurable 60-second inactivity watchdog per output. The command's delivery expiry is separate from this monotonic-clock output watchdog, so SNTP clock corrections cannot extend the HIGH state. An applied ack confirms the GPIO software latch write, not independently measured physical voltage or load action. Status is retained and refreshed every second with verified software A/B states.

Tasks, memory and timing

Task	Priority	Stack bytes	Behavior
vAnalogAcquisitionTask	5	4096	acquire six native channels every 100ms
vMqttPublishTask	4	6144	latest sample every 100ms; control watchdog
MQTT library task	library default	6144	transport/event callbacks
Wi-Fi reconnect worker	3	3072	reconnect with 1s..30s bounded backoff
MQTT reconnect worker	3	3072	reconnect with 1s..30s bounded backoff

Acquisition and publication use `vTaskDelayUntil()`. The statically allocated four-sample queue drops its oldest sample when full and counts losses. The publisher drains to the newest sample and counts skipped samples. While offline samples are discarded and counted, rather than buffered indefinitely. A repeated sample keeps its original sequence and acquisition timestamps and sets `stale`. The MQTT library owns copied payloads in its bounded 16KiB outbox; expired or unacknowledged messages can be lost. No local flash telemetry log is provided. Maximum outbox residence is configured to 5s. Resources log free heap and the publisher stack high-water mark every second; these are diagnostic observations, not a completed long-duration hardware stability test.

Static buffers hold a 2048-byte telemetry JSON payload, a 512-byte fragmented command, an eight-command queue and a 64-command deduplication cache. The command parser allocates through cJSON only on incoming commands; the periodic ADC loop allocates nothing. ESP-MQTT may allocate bounded outbox objects during enqueue. All SDK and third-party allocation is a documented coding-standard deviation. This project follows defensive C principles; it has not undergone a formal MISRA/CERT conformance analysis.

Network and command protocol

All topics are `navsphere/<device_id>/<suffix>`:

Set `CONFIG_NAV_MQTT_TOPIC_PREFIX` in menuconfig to change `navsphere` (1..64 letters, digits, underscores or hyphens). Match backend/simulator `MQTT_TOPIC_PREFIX` and the broker ACL when changing it. The default suffixes

define the versioned protocol below. If lowering the firmware's maximum command TTL, lower the backend command timeout to the same value or less.

Suffix	Device behavior	QoS / retained
telemetry	publishes schema v1	1 / no
command	subscribes strict A/B requests	1 / rejected if retained
ack	accepted, then applied or rejected	1 / no
status	online, source, boot_id, A/B states; offline LWT	1 / yes

Telemetry fields match the backend contract: schema_version, device_id, boot_id, sequence, nullable timestamp_ms, time_valid, uptime_ms, six millivolt channels, six raw counts, valid_mask, mqtt_connected, dropped_samples, source=hardware, stale. SNTP synchronizes epoch time; before a plausible epoch is available the timestamp is null and commands are rejected. `boot_id` changes on every boot, so restarting sequence zero does not collide with earlier measurements. Sequence is uint32; reboot before wrap (about 13.6 years at 10Hz).

A command must contain exactly these five keys, without duplicate fields:

```
{"command_id":"12345678-abcd-1234-abcd-
  123456789abc","key":"A","value":1,"issued_at_ms":1790000000000,"expires_at_ms":1790000010000}
```

Command IDs must have UUID syntax. Keys A/B and integer values 0/1 only. Maximum delivery TTL is 30s; future issue times tolerate at most 2s of clock skew. Malformed/truncated/oversized/embedded-NUL JSON is rejected. Full validation precedes GPIO updates. The eight-command queue bounds callback work. A repeated command is rejected without execution, including after state reset. Cache entries stay until at least 60s after expiry; a full cache refuses new commands instead of evicting a live replay guard. Invalid messages without a valid ID cannot be correlated and produce no ack. Broker authentication and device-scoped ACLs authorize senders; there is no application-layer command signature.

```
{"command_id":"12345678-abcd-1234-abcd-
  123456789abc","key":"A","value":1,"status":"applied","reason":"gpio_latch_applied","boot_id":"devi
  ce-boot-uuid"}
```

Build, provision and flash

From repository root on this machine:

```
powershell -NoProfile -ExecutionPolicy Bypass -File firmware/tools/build.ps1
gcc -std=c11 -Wall -Wextra -Werror -Ifirmware/main firmware/main/command_rules.c
  firmware/test/test_command_rules.c -o firmware/test/command_rules_test.exe
firmware/test/command_rules_test.exe
powershell -NoProfile -ExecutionPolicy Bypass -File firmware/tools/test_host.ps1
```

The build helper sets environment variables only in its own process and uses the existing IDF 6.0.2 tool installation. `-ExecutionPolicy Bypass` is scoped to that process and does not change machine policy. On another installation, activate the matching IDF terminal and run `idf.py set-target esp32`, `idf.py menuconfig`, `idf.py build` directly. The board is ESP32-WROOM, not S3.

Provision environment values in a private terminal (never commit them): NAV_WIFI_SSID, NAV_WIFI_PASSWORD, NAV_MQTT_URI (mqtt://192.168.9.59:1883 or a configured mqtts endpoint), NAV_MQTT_USERNAME, NAV_MQTT_PASSWORD, NAV_DEVICE_ID. Run `python firmware/tools/provision_nvs.py` using ESP-IDF Python. It writes ignored `.local/nvs.bin` using a temporary CSV, then removes the CSV. The binary itself contains secrets and

must remain private. NVS is not encrypted in the development build. Use flash/NVS encryption and secure boot for deployed devices; do not enable irreversible eFuse operations without a reviewed process.

The checked partition table reserves nvs at 0x9000, length 0x6000. Back up any existing device NVS before replacing it. Once wiring and serial port are verified, from an IDF terminal in firmware run `idf.py -p COMx flash`, then `python -m esptool --chip esp32 --port COMx write-flash 0x9000 .local/nvs.bin`, then `idf.py -p COMx monitor`. These are explicit device-writing operations; no board was flashed automatically during this implementation.

Set `CONFIG_NAV_REQUIRE_TLS=y` for protected deployments. TLS uses ESP-IDF's trusted root bundle and hostname validation; the broker certificate must chain to an included root (a private CA requires adding that CA to the bundle in menuconfig). Plaintext MQTT is for trusted local development only. Empty or missing MQTT usernames/passwords fail closed. Missing NVS leaves acquisition running and physical controls LOW without network startup.

Verification and remaining hardware checks

Host unit tests cover command ID/value/key/time/TTL/retained restrictions, strict JSON parsing (duplicate keys, booleans and fractional values), duplicate execution protection, disconnect safety, watchdog expiry and queue saturation. They compile the actual command-handler source with small host FreeRTOS queue stubs; GPIO electrical behavior remains a board test. Build checks validate native ADC1 mode when reported in the project testing report. Real hardware checks remain: ADC linearity and calibration against a meter, 10Hz timing distribution, power/reset behavior, Wi-Fi outage recovery, actual output voltage, heap/stack drift and a 24h soak. A compiled binary alone does not establish these hardware properties.

Primary references

- ESP32 ADC overview
- ESP32 ADC oneshot driver
- ESP32 ADC calibration
- ESP-MQTT configuration and nonblocking enqueue

All cited APIs were also checked against the installed IDF 6.0.2 headers.